Distributed Quantum Services for Hybrid Quantum Optimization

Soham Bhoir

Compact research note; working prototype and evaluation plan
Draft generated from Markdown on June 5, 2026

Author: Soham Bhoir **Status:** Working prototype, systems research draft, and evaluation plan **Project:** Distributed Quantum Services

Abstract

Near-term quantum workflows are constrained not only by qubit count, device noise, queue access, and backend availability, but also by the software architecture used to coordinate hybrid quantum-classical computation. Most practical workflows still assume a centralized model: a circuit is constructed locally, submitted to one backend, executed, and returned as a final result. This project studies a different systems model: **distributed quantum services**, where quantum and quantum-adjacent capabilities are advertised as discoverable peer-to-peer services, and a coordinator plans, reserves, routes, executes, and records workloads across a service network.

The prototype treats a quantum workflow as an inspectable execution graph rather than an opaque backend call. A submitted circuit or finance workload is normalized into an internal representation, converted into a dependency-aware directed acyclic graph, partitioned into fragments, mapped to candidate service peers through a fidelity- and latency-aware cost model, reserved before invocation, executed with timeout/retry/fallback behavior, and reconstructed into both execution metadata and quantum-analysis outputs.

The main research workload is mean-variance portfolio optimization. The finance problem is encoded as a Quadratic Unconstrained Binary Optimization problem, mapped to an Ising Hamiltonian, and solved using QAOA with a feasible Hamming-weight-preserving initialization and a budget-preserving XY mixer. The project is therefore both a **distributed systems prototype** and a **hybrid quantum runtime-accounting study**.

The central research question is not whether peer-to-peer orchestration alone creates quantum advantage. It is whether distributed orchestration can make hybrid quantum workloads more measurable, scalable, fault-aware, and scientifically honest. Early profiling shows that for QAOA-style workloads, classical parameter search can dominate total runtime, sharply limiting the speedup obtainable from distributing only circuit execution. This makes the platform valuable as a measurement framework: it exposes where distributed execution helps, where it does not, and which parts of the hybrid workflow must be optimized before claiming advantage.

Keywords: distributed quantum services, hybrid quantum optimization, QAOA, QUBO, Ising Hamiltonian, portfolio optimization, peer-to-peer systems, libp2p, circuit fragmentation, runtime orchestration, quantum workflow accounting

Research Positioning

This project should be understood as a **coordination layer for distributed quantum services**. It is not merely a parser, simulator, or finance demo. Its core idea is that quantum capabilities can be exposed as remotely invocable services, discovered through a network, planned against using a cost model, and executed with persistent runtime telemetry.

A service peer may advertise support for a gate, a higher-level quantum operation, a simulator-backed execution service, a future hardware-backed backend, or an auxiliary quantum-adjacent capability. The coordinator builds a live view of this service network, compiles submitted workloads into dependency-aware execution plans, assigns fragments to peers, and returns both the final quantum interpretation and the full execution trace.

The project deliberately avoids overstating its current scope. In the current proof-of-concept, **distributed quantum services** does not mean a complete quantum internet, hardware-level entanglement routing, or a production-grade multi-QPU cloud. It means:

- quantum capabilities are represented as service advertisements;
- each service is associated with a node identity, reachable address, and quality profile;
- planning is performed against a distributed service registry;
- invocation happens through a peer-to-peer transport layer;
- the coordinator can route different fragments to different service nodes;
- every major lifecycle event is persisted for analysis and reproducibility.

This framing is important for research communication. The strongest claim is not immediate quantum advantage. The strongest claim is that **hybrid quantum computation needs a measurable distributed runtime**, especially when quantum execution is only one part of a larger classical-quantum workflow.

Core Thesis

The thesis of the project is:

> *Hybrid quantum workloads should be orchestrated as distributed, measurable services rather than opaque calls to a single backend.*

This thesis has two parts: a systems thesis and an algorithmic thesis.

Systems Thesis

A credible distributed quantum runtime should separate the workflow into three planes.

The **control plane** decides what should happen. It handles API ingestion, circuit normalization, service discovery, registry freshness, dependency analysis, planning, assignment, reservation decisions, and job lifecycle management.

The **execution plane** makes the distributed work happen. It handles service advertisements, peer-to-peer request streams, fragment invocation, runtime scheduling, timeout handling, retry logic, fallback selection, and degraded-quality behavior.

The **data plane** makes the workflow inspectable and recoverable. It stores job state, execution plans, reservation events, execution events, peer capability snapshots, topology projections, benchmark results, provenance bundles, structured logs, and quantum-analysis results.

This separation matters because a quantum experiment should not only return a bitstring or probability distribution. It should also answer:

- which nodes executed which fragments;
- which assignments were chosen and why;
- which reservations succeeded or failed;

- how many attempts were required;
- what fidelity or quality values were observed;
- where time was spent;
- whether the final quantum interpretation matches the intended circuit.

Algorithmic Thesis

For hybrid algorithms, the relevant performance question is not simply:

> *How long did the quantum circuit take to run?*

The more useful question is:

> *What fraction of end-to-end runtime was spent in data ingestion, QUBO construction, parameter search, circuit construction, planning, reservation, network dispatch, execution, reconstruction, and classical comparison?*

This distinction is critical because distributed execution only accelerates the parallelizable part of the workflow. If classical parameter search dominates the total runtime, distributing the final circuit execution may have only limited impact.

System Architecture

The prototype can be summarized as the following pipeline:

Code / workflow sketch

```
client workload
-> API ingestion
-> circuit or problem normalization
-> dependency DAG construction
-> fragmentation
-> service discovery and registry lookup
-> cost-based planning
-> reservation
-> peer-to-peer invocation
-> fragment result persistence
-> quantum result reconstruction
-> benchmark and comparison reporting
```

A simplified architecture view is:

Mermaid diagram source

```
flowchart TB
    Client[Researcher or API Client]
    API[FastAPI Layer]
    JobManager[Job Manager]
    Parser[Circuit / Problem Normalizer]
    DAG[Dependency DAG Builder]
    Planner[Distributed Planner]
    Registry[Service Registry]
    Reservation[Reservation Protocol]
    Runtime[Runtime Executor]
    Fabric[py-libp2p Fabric]
    PeerA[Service Peer A]
    PeerB[Service Peer B]
    PeerC[Service Peer C]
    Store[(Persistence Layer)]
    Qiskit[Qiskit Result Builder]
```

```
Client --> API
API --> JobManager
JobManager --> Parser
Parser --> DAG
DAG --> Planner
Planner --> Registry
Planner --> Reservation
JobManager --> Runtime
Runtime --> Fabric
Fabric --> PeerA
Fabric --> PeerB
Fabric --> PeerC
Runtime --> Store
Registry --> Store
Reservation --> Store
Runtime --> Qiskit
Qiskit --> Store
Store --> API
```

The coordinator owns the lifecycle of a submitted job:

Code / workflow sketch

```
QUEUED -> COMPILING -> RESERVING -> EXECUTING -> COMPLETED | FAILED
```

A reservation follows a separate lifecycle:

Code / workflow sketch

```
REQUESTED -> PREPARED -> COMMITTED -> EXECUTED
                         -> EXPIRED
                         -> CANCELED
REQUESTED -> REJECTED
```

Separating the job lifecycle from the reservation lifecycle prevents the runtime from pretending that planning and execution are the same problem. Planning answers, "where should this fragment go?" Reservation answers, "can this peer still execute it under the requested constraints right now?"

Service Model

Each peer advertises a capability profile. At minimum, a useful profile contains:

Code / workflow sketch

```
node_id
listen_addrs
service_type
fidelity or quality estimate
qubit_min
qubit_max
availability
updated_at
protocol_version
```

The coordinator maintains a freshness-aware local registry. The planner does not make a network call for every candidate decision; it plans against the registry snapshot. This makes planning deterministic for a fixed topology and configuration, which is important for repeatable experiments.

For a fragment \(f_i\), the feasible peer set is:

```
\mathcal{P}(f_i)=\{p_j \mid p_j \text{ supports } f_i \text{ and satisfies
    quality and capacity constraints}\}.
```

Candidate mappings are scored using a configurable cost model:

```
\operatorname{cost}(f_i,p_j)
=
w_{\mathrm{lat}}L(f_i,p_j)
+
w_{\mathrm{fail}}R(f_i,p_j)
+
w_{\mathrm{ent}}E(f_i,p_j)
+
w_{\mathrm{load}}D(p_j).
```

Here:

- \(L\) represents expected latency or orchestration delay;
- \(R\) represents failure risk, often derived from fidelity and timeout history;
- \(E\) represents entanglement or remote-link overhead, where modeled;
- \(D\) represents node load or congestion;
- the \(w\) values are tunable planning weights.

The planner selects a primary peer and one or more fallback peers when available. The runtime then reserves and invokes the selected peer. If reservation fails, invocation times out, connectivity drops, or fidelity falls below threshold, the runtime can retry, move to a fallback, or fail with a fragment-level reason.

Circuit Normalization, Dependency Graphs, and Fragmentation

The planner does not operate directly on raw OpenQASM text. It first normalizes the input into a small internal representation that records:

- circuit format;
- qubit count;
- ordered operations;
- service type per operation;
- qubit operands;
- measurement information where applicable.

Supported input styles include OpenQASM-style circuit declarations and project-specific service aliases such as Bell-pair generation, teleportation-style operations, and measurement/feedforward services.

The dependency graph is built from qubit usage. If two operations touch the same qubit, their relative order must be preserved. This produces a directed acyclic graph:

```
G=(V,E),
```

where each vertex \(v\in V\) is an operation or fragment, and each edge \((u,v)\in E\) means that \(v\) depends on \(u\).

A simplified dependency rule is:

Code / workflow sketch

```
for each operation op:
    for each qubit q touched by op:
        if q has a previous operation prev:
            add edge prev -> op
        latest[q] = op
```

Fragments are then formed from the DAG. The current implementation should be described conservatively: fragments may be operation-level or small block-level units depending on the workload and planning path. A fragment carries:

Code / workflow sketch

```
fragment_id
service_type
qubits
operation_ids
dependencies
primary_assignment
fallback_assignments
quality_snapshot_id
```

This is preferable to claiming that the system always uses one operation per fragment. Operation-level fragmentation is easy to inspect, but larger blocks can reduce orchestration overhead. The research-relevant point is that the runtime executes **ready fragments whose dependencies are satisfied**, rather than treating the whole circuit as one indivisible backend call.

Portfolio Optimization Workload

The main research workload is mean-variance portfolio optimization. This problem is a good first workload because it has:

- a clear binary decision structure;
- a natural QUBO formulation;
- a direct Ising mapping;
- meaningful constraints;
- interpretable outputs;
- classical baselines for comparison.

Given:

- N assets;
- expected return vector $\boldsymbol{\mu}\in\mathbb{R}^{N}$;
- covariance matrix $\Sigma\in\mathbb{R}^{N\times N}$;
- binary decision vector $\mathbf{x}\in\{0,1\}^{N}$;
- budget B, meaning exactly B assets should be selected;
- risk-aversion parameter $q>0$;

we maximize risk-adjusted return:

$$
\max_{\mathbf{x}\in\{0,1\}^{N}}
\quad
\boldsymbol{\mu}^{T}\mathbf{x}
-
q\mathbf{x}^{T}\Sigma\mathbf{x}
$$

subject to:

$$
\mathbf{1}^{T}\mathbf{x}=B.
$$

Equivalently, the minimization objective is:

$$
\min_{\mathbf{x}}
\quad
q\mathbf{x}^{T}\Sigma\mathbf{x}
-
\boldsymbol{\mu}^{T}\mathbf{x}.
$$

The budget constraint is folded into the objective with a quadratic penalty:

$$
H_{\mathrm{QUBO}}(\mathbf{x})
=
q\mathbf{x}^{T}\Sigma\mathbf{x}
-
\boldsymbol{\mu}^{T}\mathbf{x}
+
P\left(\sum_i x_i-B\right)^2,
$$

where $P>0$ should be large enough that violating the budget is worse than improving the unconstrained objective.

QUBO Coefficients and Ising Mapping

Using the upper-triangular pairwise convention

$$
H_{\mathrm{QUBO}}(\mathbf{x})
=
c+
\sum_i a_i x_i+
\sum_{i<j} b_{ij}x_i x_j,
$$

the coefficients are:

$$
a_i
=
q\Sigma_{ii}-\mu_i+P(1-2B),
$$

$$
b_{ij}
=
2q\Sigma_{ij}+2P,
\quad i<j,
$$

$$
c=PB^2.
$$

This convention assumes that off-diagonal covariance terms are collected once as $i<j$, while the symmetric contribution from $\mathbf{x}^{T}\Sigma\mathbf{x}$ appears as $2q\Sigma_{ij}$. If an implementation stores a full symmetric QUBO matrix instead of upper-triangular pair terms, the coefficients must be adjusted consistently. Otherwise diagonal risk terms can accidentally receive an extra factor.

Binary variables are mapped to Pauli-Z operators by:

$$
x_i=\frac{1-Z_i}{2}.
$$

Substituting into the QUBO gives an Ising cost Hamiltonian:

$$
H_C
=
C+
\sum_i \alpha_i Z_i+
\sum_{i<j}\beta_{ij}Z_iZ_j,
$$

where:

$$
\alpha_i
=
-\frac{a_i}{2}
$$

```
-
\frac{1}{4}\sum_{j\neq i}b_{\min(i,j),\max(i,j)},

\beta_{ij}=\frac{b_{ij}}{4},
```

and

```
C
=
c+
\frac{1}{2}\sum_i a_i+
\frac{1}{4}\sum_{i<j}b_{ij}.
```

The constant \(C\) shifts all energies equally and does not affect which bitstring minimizes the objective, but it is useful when comparing objective values against classical baselines.

QAOA Circuit Construction

For QAOA depth \(p\), the parameterized state is:

```
|\psi(\boldsymbol{\beta},\boldsymbol{\gamma})\rangle
=
\prod_{\ell=1}^{p}
U_M(\beta_\ell)U_C(\gamma_\ell)
|s_0\rangle.
```

The cost unitary is:

```
U_C(\gamma)=e^{-i\gamma H_C}.
```

A standard X-mixer can move probability mass into infeasible bitstrings with the wrong number of selected assets. For a cardinality-constrained portfolio problem, the prototype therefore uses a budget-preserving XY-style mixer:

```
H_M^{XY}
=
\sum_{(i,j)\in\mathrm{ring}}
\left(X_iX_j+Y_iY_j\right),
```

with mixer unitary:

```
U_M(\beta)
=
\prod_{(i,j)\in\mathrm{ring}}
\exp\left[-i\frac{\beta}{2}(X_iX_j+Y_iY_j)\right].
```

The XY interaction swaps neighboring \(|01\rangle\) and \(|10\rangle\) amplitudes while preserving Hamming weight. Therefore, if the initial bitstring has exactly \(B\) selected assets, the mixer keeps the search inside the feasible budget subspace.

The initial state is a feasible greedy bitstring:

```
|s_0\rangle=|\mathbf{x}_{\mathrm{greedy}}\rangle,
\quad
\sum_i x_{\mathrm{greedy},i}=B.
```

A conservative implementation can choose the top \(B\) assets by expected return \(\mu_i\). A risk-adjusted variant can choose the top \(B\) assets by:

```
\mathrm{score}_i=\mu_i-q\Sigma_{ii}.
```

Either choice should be described as a heuristic warm start, not as an optimality assumption. Its role is to initialize QAOA inside the feasible subspace and reduce wasted exploration.

Distributed Runtime Semantics

The runtime result has two layers of meaning.

The first layer is **execution truth**. This includes:

- fragment identifiers;
- selected peer IDs;
- reservation outcomes;
- attempt counts;
- start and finish timestamps;
- observed quality or fidelity;
- timeout, retry, fallback, or failure reasons.

The second layer is **quantum interpretation**. This includes:

- counts;
- probability distributions;
- measured-qubit marginal probabilities;
- statevector or simulated state summary where available;
- observable expectations;
- reduced density matrices;
- Bloch vectors;
- entanglement entropy;
- top basis states;
- final fidelity or consistency estimates.

This split is central to the project. A distributed quantum runtime should answer both:

> *Did the distributed network execution succeed?*

and:

> *What quantum state, measurement distribution, or optimization result does that execution imply?*

In the current simulator-backed prototype, any state handoff or statevector reconstruction should be treated as a simulation/runtime abstraction, not as a claim that physical quantum state is being transferred between real QPUs. This distinction makes the research framing more accurate and credible.

Methodology

The methodology is organized around correctness, runtime accounting, baseline comparison, and failure behavior.

End-to-End Workflow Validation

The first objective is to verify the complete workflow:

Code / workflow sketch

```
dataset or circuit
-> mathematical encoding
-> circuit construction
-> dependency planning
-> fragmentation
-> peer assignment
-> reservation
-> distributed execution
-> result reconstruction
-> classical comparison
```

Correctness is measured by comparing distributed reconstructed results against local full-circuit simulation where feasible, and by comparing optimization outputs against classical baselines for small problem instances.

Runtime Accounting

The platform should report the full hybrid runtime, not only circuit execution. Important timing components include:

Code / workflow sketch

```
data ingestion
QUBO construction
Ising mapping
QAOA parameter search
circuit construction
planning
reservation
distributed dispatch
fragment execution
result aggregation
quantum result reconstruction
classical baseline runtime
comparison report generation
```

This accounting prevents misleading performance claims. A workflow can look fast if only circuit execution is reported while parameter optimization, compilation, and post-processing are ignored.

Classical Baselines

The quantum workflow should be compared against appropriate classical baselines, including:

- exact enumeration for small asset counts;
- greedy heuristics;
- simulated annealing for larger asset counts;
- mixed-integer or quadratic-programming baselines where available;
- local Qiskit/statevector execution for correctness comparison.

The comparison should include both runtime and solution quality:

Code / workflow sketch

```
objective value
selected assets
constraint satisfaction
feasible probability mass
approximation ratio
objective gap
```

```
return gap
variance or risk gap
runtime breakdown
```

Peer Scaling and Fault Injection

The distributed runtime should be evaluated across different peer counts, for example:

$$N_{\mathrm{peers}}\in\{1,5,10,20,50,100\}.$$

The relevant fault cases include:

- node drop;
- reservation rejection;
- timeout;
- connection reset;
- delayed peer response;
- degraded fidelity;
- malformed fragment response;
- missing capability;
- stale service advertisement.

The important metrics are success rate, fallback rate, additional latency, observed quality, final reconstruction consistency, and whether the failure mode is visible in persisted telemetry.

Preliminary Findings

The key preliminary finding is that QAOA parameter search can dominate total runtime. This matters because peer scaling only accelerates the part of the workflow that is actually parallelized.

Let s be the serial fraction of the workflow and N be the number of parallel peers. Amdahl's Law gives:

$$S(N)=\frac{1}{s+\frac{1-s}{N}}.$$

If parameter search, circuit construction, planning, or result reconstruction form a large serial fraction, then even a large peer network provides only modest speedup. In some benchmark configurations, parameter search accounted for the majority of quantum runtime, while distributed execution represented only a smaller parallelizable component. This explains why increasing peer count can produce diminishing returns.

The most important interpretation is:

> *A distributed quantum runtime is only as useful as the workload decomposition it exposes.*

For QAOA portfolio optimization, the platform shows that distributing final circuit execution is not enough. The full hybrid loop must be measured, especially the classical optimizer, the number of circuit evaluations, objective-function cost, simulation cost, and quality/runtime tradeoff.

This should be presented as a strength of the project. The system does not hide negative results. It reveals them.

Scope Boundaries and Current Limitations

The current prototype should be described carefully. Its limitations include:

- simulator-backed or mocked quantum operations rather than real hardware execution;
- no production-grade multi-QPU hardware scheduler;
- no complete quantum internet or entanglement-routing stack;
- no full production security or multi-tenant authorization model;
- simplified fidelity and quality modeling;
- limited hardware topology modeling;
- statevector simulation artifacts that should not be confused with hardware behavior;
- conservative fragmentation and orchestration strategies;
- parameter-search bottlenecks in QAOA-style workloads;
- need for broader repeated trials, confidence intervals, and controlled datasets;
- need for stronger classical baselines for publishable claims.

These limitations do not weaken the research story. They define the honest boundary of the current system. The project is strongest when framed as a rigorous systems-and-algorithms investigation rather than a premature claim of quantum speedup.

Research Contribution

The contribution of the project can be summarized in four points.

First, it proposes a service-oriented runtime model for hybrid quantum workloads. Instead of treating a backend as a single opaque endpoint, it treats capabilities as discoverable services with identities, quality metadata, availability, and invocation semantics.

Second, it connects distributed systems concerns to quantum workflow execution. The project includes planning, peer discovery, reservation, retry, fallback, runtime telemetry, persistence, and post-mortem analysis.

Third, it applies the runtime to a concrete optimization workload: mean-variance portfolio optimization encoded as QUBO, mapped to an Ising Hamiltonian, and solved with QAOA using a feasible initialization and Hamming-weight-preserving mixer.

Fourth, it emphasizes full workflow accounting. The project explicitly measures the parts of the hybrid computation that are often ignored: parameter optimization, compilation, planning, distributed orchestration, reconstruction, and classical comparison.

This combination makes the project relevant to researchers working on quantum software systems, hybrid algorithms, distributed runtimes, quantum benchmarking, and quantum finance workflows.

Conclusion

This draft presents Distributed Quantum Services as a runtime model for hybrid quantum optimization. The prototype represents quantum capabilities as network services, compiles workloads into dependency-aware plans, assigns fragments to peers through a cost model, reserves and executes distributed work with fallback behavior, and reconstructs quantum results with persistent execution metadata.

The portfolio-optimization workload provides a concrete algorithmic test case. It connects financial optimization, QUBO construction, Ising Hamiltonians, QAOA, feasible warm starts, and budget-preserving mixers to a distributed execution substrate.

The main technical lesson is that distributed quantum systems must be evaluated at the level of the full hybrid workflow. For QAOA-style optimization, classical parameter search can dominate runtime and limit parallel speedup. The platform's value lies in making that bottleneck visible, measurable, and comparable against classical baselines. In that sense, the project is not just a distributed quantum demo. It is a framework for honest hybrid quantum systems research.

Appendix: Notation

Symbol	Meaning
N	Number of assets / qubits
B	Budget or cardinality constraint
\mathbf{x}	Binary asset-selection vector
\boldsymbol{\mu}	Expected return vector
\Sigma	Asset-return covariance matrix
q	Risk-aversion parameter
P	Constraint-penalty coefficient
H_{\mathrm{QUBO}}	QUBO objective Hamiltonian
H_C	QAOA cost Hamiltonian
H_M^{XY}	XY mixer Hamiltonian
p	QAOA depth / number of layers
\beta,\gamma	QAOA variational parameters
G=(V,E)	Dependency graph
f_i	Circuit fragment
p_j	Service peer
s	Serial fraction in Amdahl's Law